

3. (8 punti) Per la funzione definita precedentemente e data l'implementazione sottostante, definire i test case per ottenere le più alte node coverage, edge coverage, multiple condition coverage, loop coverage, and path coverage.

Per i test, scrivere solo il valore di input.

```

1. class PrimeChecker {
2.
3.     public static isPrime(numero: number): boolean {
4.
5.         if (numero < 2) {
6.             return false;
7.         }
8.
9.         for (let i = 2; i <= Math.sqrt(numero); i++) {
10.            if (numero % i === 0) {
11.                return false;
12.            }
13.        }
14.
15.        return true;
16.    }
17.
18. }

```

| Tipo Coverage | Numero di casi di test necessari per ottenere il 100% di copertura                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Copertura ottenuta con i casi di test definiti (%) | Casi di test definiti                                                                   |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------|
| Statement     | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 100                                                | T1 (linea 5,6)<br>T2 (linea 5, 9, 15)<br>T3 (linea 5, 9, 10, 11)                        |
| Decision      | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 100                                                | Linea 5: T1 true, T2 false<br>Linea 10: T2 true, T7 false (e true)<br><br>T7(77, false) |
| Loop          | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 100                                                | 0 Loop: T2<br>1 Loop: T3<br>N Loop: T7                                                  |
| Path          | <p>Il percorso dei numeri negativi 1-5-6 può essere testato da un solo numero negativo.</p> <p>I percorsi positivi possono essere suddivisi in due schemi:</p> <ol style="list-style-type: none"> <li>1. [9-10] n volte, 11 (non primo)</li> <li>2. [9-10] n volte, 15 (primo)</li> </ol> <p>Utilizzando brute force, questi percorsi vengono provati utilizzando tutti i numeri positivi da 1 al number più alto.</p> <p>Ma i non primi che hanno lo stesso fattore (ad esempio 4 e 6 e 8, o 9, 12, 15) hanno lo stesso percorso; quindi, se testiamo il 4 possiamo evitare di testare tutti i numeri successivi con il 2 come fattore, e così via.</p> |                                                    |                                                                                         |

Per la seguente funzione definire i casi di test per ottenere la massima copertura possibile di nodi, bordi, condizioni multiple, cicli e percorsi. Per i casi di test, scrivere solo il valore di ingresso.

```

1 function doubleAverage(grade1: number, grade2: number, grade3: number, grade4: number, grade5: number, grade6: number): number {
2   let grades: number[] = [grade1, grade2, grade3, grade4, grade5, grade6];
3   let sum: number = 0.0;
4   let min: number = 33;
5   let max: number = 0;
6   for (let i = 0; i < 6; i++) {
7     sum += grades[i];
8     if (grades[i] < min) min = grades[i];
9     if (grades[i] > max) max = grades[i];
10  }
11  return (sum - max - min) / 4;
12 }

```

| Coverage type               | Feasibility (Y/N) | Coverage obtained (%)                                                     | Test cases |
|-----------------------------|-------------------|---------------------------------------------------------------------------|------------|
| node                        | Y                 | 100                                                                       | T1         |
| edge                        | Y                 | 100                                                                       | T1         |
| multiple condition (line 8) | -                 | There is no multiple condition at line 8, t1 achieves 100%                | T1         |
| loop                        | Not controllable  | 33                                                                        | T1         |
| path                        | Y                 | There are $4^6$ possible paths, so in principle 100% coverage is feasible |            |

**T1 (2, 30, 25, 26, 18, 23)**

Per la seguente funzione, definire:

- control flow graph
- definire i test cases per ottenere la più alta node coverage
- edge coverage
- multiple condition coverage
- loop coverage
- path coverage

```

1 function racerDisqualified(times: number[], winnerTimes: number[], nPenalties: number, penalties: number[]): boolean {
2
3   let disqualified = false;
4   let maxTime: number;
5   let totPenalties = 0;
6
7   for (let i = 0; i < nPenalties; i++) {
8     totPenalties += penalties[i];
9     if (penalties[i] > 100) {
10      disqualified = true;
11    }
12  }
13
14  if (totPenalties > 100 || nPenalties > 5) {
15    disqualified = true;
16  }
17
18  for (let i = 0; i < 3; i++) {
19    maxTime = winnerTimes[i] * 1.5;
20    if (times[i] > maxTime) {
21      disqualified = true;
22    }
23  }
24
25  return disqualified;
26 }

```

| Coverage Type | Numero di test case necessari per raggiungere il 100% di coverage | Coverage ottenuta con i test case definiti (%) | Test case definiti |
|---------------|-------------------------------------------------------------------|------------------------------------------------|--------------------|
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |
|               |                                                                   |                                                |                    |

Scrivere solo l'id del test case (T1, T2 ..) nella colonna più a destra della tabella, e i veri test cases qui

3 For the function above and the implementation below, define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.  
For the test cases, **write only the input value**.

```

1. class PrimeChecker {
2.
3.   public static isPrime(numero: number): boolean {
4.
5.     if (numero < 2) {
6.       return false;
7.     }
8.
9.     for (let i = 2; i <= Math.sqrt(numero); i++) {
10.      if (numero % i === 0) {
11.        return false;
12.      }
13.    }
14.
15.    return true;
16.  }
17.
18. }

```

| Coverage type | Number of test cases needed to | Coverage obtained | Test cases defined |
|---------------|--------------------------------|-------------------|--------------------|
|---------------|--------------------------------|-------------------|--------------------|

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                             |                                                                                         |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|-----------------------------------------------------------------------------------------|
|           | obtain 100% coverage                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | with test cases defined (%) | <div>Ask AI</div>                                                                       |
| statement | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 100                         | T1 (line 5,6)<br>T2 (line 5, 9, 15)<br>T3 (line 5, 9, 10, 11)                           |
| decision  | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 100                         | Line 5: T1 true, T2 false<br>Line 10: T2 true, T7 false (and true)<br><br>T7(77, false) |
| Loop      | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 100                         | Loop zero: T2<br>Loop once: T3<br>Loop many: T7                                         |
| Path      | <p>The negative number path 1-5-6 can be tested by just one negative number</p> <p>The positive paths can be split in two patterns</p> <p>[9-10] n times, 11 (non prime)<br/>And<br/>[9-10] n times, 15 (prime)<br/>Using brute force these paths are tried using all positive numbers from 1 to highest integer, so if number is represented as integer on 32 bits, it means <math>2^{(31)}</math> test cases, roughly <math>10^{10}</math>, making path coverage unfeasible in practice.</p> <p>However non primes having the same factor (ex 4 and 6 and 8, or 9, 12, 15) have the same path, so if we test 4 we could avoid testing all following numbers with 2 as a factor, if we test 9 we could avoid all following numbers having 3 as a factor and so on.</p> <p>In all cases 100% path coverage is hardly feasible.</p> |                             |                                                                                         |

Typical error. A test like T1(-10) does NOT tries the loop without entering (zero times loop) because the flow exits at line 6. T2 flows till line 9, tries the loop, but does not enter it.

3 For the function above and the implementation below, define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.  
For the test cases, **write only the input value.**

```

1. class PrimeChecker {
2.
3.   public static isPrime(numero: number): boolean {
4.
5.     if (numero < 2) {
6.       return false;
7.     }
8.
9.     for (let i = 2; i <= Math.sqrt(numero); i++) {
10.      if (numero % i === 0) {
11.        return false;
12.      }
13.    }
14.
15.    return true;
16.  }
17.
18. }

```

| Coverage type | Number of test cases needed to | Coverage obtained | Test cases defined |
|---------------|--------------------------------|-------------------|--------------------|
|---------------|--------------------------------|-------------------|--------------------|



|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                             |                                                                                         |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|-----------------------------------------------------------------------------------------|
|           | obtain 100% coverage                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | with test cases defined (%) |                                                                                         |
| statement | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 100                         | T1 (line 5,6)<br>T2 (line 5, 9, 15)<br>T3 (line 5, 9, 10, 11)                           |
| decision  | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 100                         | Line 5: T1 true, T2 false<br>Line 10: T2 true, T7 false (and true)<br><br>T7(77, false) |
| Loop      | 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 100                         | Loop zero: T2<br>Loop once: T3<br>Loop many: T7                                         |
| Path      | <p>The negative number path 1-5-6 can be tested by just one negative number</p> <p>The positive paths can be split in two patterns</p> <p>[9-10] n times, 11 (non prime)<br/>And<br/>[9-10] n times, 15 (prime)<br/>Using brute force these paths are tried using all positive numbers from 1 to highest integer, so if number is represented as integer on 32 bits, it means <math>2^{(31)}</math> test cases, roughly <math>10^{10}</math>, making path coverage unfeasible in practice.</p> <p>However non primes having the same factor (ex 4 and 6 and 8, or 9, 12, 15) have the same path, so if we test 4 we could avoid testing all following numbers with 2 as a factor, if we test 9 we could avoid all following numbers having 3 as a factor and so on.</p> <p>In all cases 100% path coverage is hardly feasible.</p> |                             |                                                                                         |

Typical error. A test like T1(-10) does NOT tries the loop without entering (zero times loop) because the flow exits at line 6. T2 flows till line 9, tries the loop, but does not enter it.

For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.  
For the test cases, **write only the input value.**

```

1  private int compute_tax(int wage) {
2
3      int ranges[] = {5000,15000,30000};
4      int amount_due = 0;
5      int level=0;
6      boolean control = true;
7      for(int i=0;i<3;i++){
8          if(wage>ranges[i] )
9              level++;
10     }
11     if(level==1 && control == true)
12         amount_due = 500;
13     else if(level == 2)
14         amount_due = 1500;
15     else if(level == 3)
16         amount_due = 3000;
17     return amount_due;
18 }

```

| Coverage type                | Feasibility (Y/N)                                                                                                                                                                                                      | Coverage obtained (%) | Test cases                         |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|------------------------------------|
| Node                         | y                                                                                                                                                                                                                      | 100                   | T1(6000)<br>T2(16000)<br>T3(35000) |
| Edge                         |                                                                                                                                                                                                                        |                       | T1 T2 T3 T4(500)                   |
| Multiple condition (line 11) | N<br>Only FT TT are possible                                                                                                                                                                                           | 50                    | T1, T2                             |
| Loop                         | N<br>No way of controlling the loop, always 3 iterations                                                                                                                                                               | 33                    | Any test T1, T2, T3                |
| Path                         | Y<br>Lines 7-10: in theory 2^3 paths, however only 4 are feasible (covered by T1 T2 T3 T4)<br><br>Lines 11-16: in theory 2^3 paths, however only 4 are feasible (again covered by T1 to T4)<br><br>Combining the cases | 100                   | T1 T2 T3 T4                        |

|  |                                                                                                                             |  |  |
|--|-----------------------------------------------------------------------------------------------------------------------------|--|--|
|  | above we have 4*4 paths, but in practice they are dependent, and only 4 are feasible overall – so path coverage is feasible |  |  |
|--|-----------------------------------------------------------------------------------------------------------------------------|--|--|

8 For the following function define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.

For the test cases, **write only the input value.**

```

1    int f (int num1, int num2, int num3)
2    {
3        if ((num1 == num2) || (num2 == num3)) printf("no ok");
4        while (num2 < 0)
5            {
6                num1 = num1 - 1;
7                num2 = num2 - 1;
8            }
9        if (num1 >= num3) printf("ok");
10       else printf("no ok");
11    }

```

| Coverage type               | Feasibility (Y/N)                                                                                                                                                                                             | Coverage obtained (%)        | Test cases                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|----------------------------------------------------------|
| node                        | Y                                                                                                                                                                                                             | 100                          |                                                          |
| edge                        | Y                                                                                                                                                                                                             | 100                          | T1(2,2,2)<br>T2(1,-2,3)<br>T3(1,2,3)                     |
| multiple condition (line 3) | Y                                                                                                                                                                                                             | 100                          | TT T1<br>TF<br>T4(2,2,3)<br>FT<br>T5(3,2,2)<br>FF T2, T3 |
| Loop (line 4)               | N                                                                                                                                                                                                             | 66 % (one loop not feasible) | Many T2<br>None T1, T3, T4 T5                            |
| path                        | Not feasible because of bug in while. Also fixing this bug the number of paths is high ( $1 + \text{num2}^2$ ) with num2 depending on the number of bits used to represent an integer (if 32 bits, $2^{31}$ ) |                              |                                                          |

A negative num2 produces an endless loop

For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.  
For the test cases, **write only the input value.**

```

1. int is_number(char A[], int n){
2.     int i;
3.     for(i=n-1; i>=0; i--){
4.         if( !( isDigit(A[i]) || isComma(A[i]) || isDot(A[i]) ) ){
5.             return 0;
6.         }
7.     }
8.     return 1;
9. }

```

| Coverage type             | Number of test cases needed to obtain 100% coverage                  | Coverage obtained with test cases defined (%)                         | Test cases defined                                                     |
|---------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------|
| Node                      | 2                                                                    | 100                                                                   | T1( ['1', '2', '3'], 3 )<br>T2( ['a', 'b', '3'], 3 )                   |
| Edge                      | 2                                                                    | 100                                                                   | T1 T2                                                                  |
| Loop line 3               | 3                                                                    | 100                                                                   | T1 loop n times<br>T3(['1'], 1) loop once<br>T4([], 0) loop zero times |
| Multiple condition line 4 | 8                                                                    | 50<br>TTT or TTF TFT FTT are not feasible<br>FFF<br>TFF<br>FTF<br>FFT | FFF T1<br>TFF T2<br>FTF T5 (['.'], 1)<br>FFT T6 (['.'], 1)             |
| Path                      | Depends on length of array, feasible only if a max length is defined | Close to zero with the 6 test cases defined                           |                                                                        |



8 For the following function define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.

For the test cases, **write only the input value.**

```

1    double average (int grade1, int grade2, int grade3, int grade4,
int grade5, int grade6){
2        int grades[6]; grades[0]= grade1; grades[1]= grade2;
3        grades[2]= grade3; grades[3]= grade4;
4        grades[4]= grade5; grades[5]= grade6;
5        double sum = 0.0; double min =33; double max = 0;
6        for (int i=0; i<6; i++){

7            sum = sum + grades[i];
8            if (grades[i] < min) min = grades[i];
9            if (grades[i] > max) max = grades[i];
10        }
11    return (sum -max - min)/4;}

```

| Coverage type               | Feasibility (Y/N) | Coverage obtained (%)                                                   | Test cases |
|-----------------------------|-------------------|-------------------------------------------------------------------------|------------|
| node                        | Y                 | 100                                                                     | T1         |
| edge                        | Y                 | 100                                                                     | T1         |
| multiple condition (line 8) | -                 | There is no multiple condition at line 8, t1 achieves 100%              | T1         |
| loop                        | Not controllable  | 33                                                                      | T1         |
| path                        | Y                 | There are 4^6 possible paths, so in principle 100% coverage is feasible |            |

**T1 (2, 30, 25, 26, 18, 23)**

3 (7 points) – For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage. For the test cases, **write only the input value**.

Write control flow graph here

```

1  int compute_tax(int wage) {
2
3      int ranges[] = {6000, 18000, 36000};
4      int amount_due = 0;
5      int level=0;
6
7      for(int i=0;i<3;i++){
8          if(wage>ranges[i])
9              level++;
10     }
11     if(level==1)
12         amount_due = 500;
13     else if(level == 2)
14         amount_due = 1500;
15     else if(level == 3)
16         amount_due = 3000;
17     return amount_due;
18 }
```

| Coverage type             | Number of test cases needed to obtain 100% coverage                                                                                                                                         | Coverage obtained with test cases defined (%) | Test cases defined |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------|
| Node                      | 3                                                                                                                                                                                           | 100%                                          | T1,T2, T3          |
| Edge                      | 4                                                                                                                                                                                           | 100%                                          | T1,T2, T3, T4      |
| Multiple condition line 8 | Not a multiple, 2 are enough (even only 1 because of for cycle line 7)                                                                                                                      | 100%                                          | T1                 |
| Loop line 7               | 3, but not controllable                                                                                                                                                                     | 33%                                           | Any input          |
| Path                      | For in line 7, in theory $2^3$ , in practice 4 only<br>Line 11 to 16: 4<br>Should be $4*4$<br>However paths in the two parts of the function are correlated, so overall 4 paths<br>Feasible |                                               |                    |

Write test case ID (t1, T2 ..) in the rightmost column, and test cases here

3 (7 points) – For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage. For the test cases, **write only the input value**.

Write control flow graph here

```

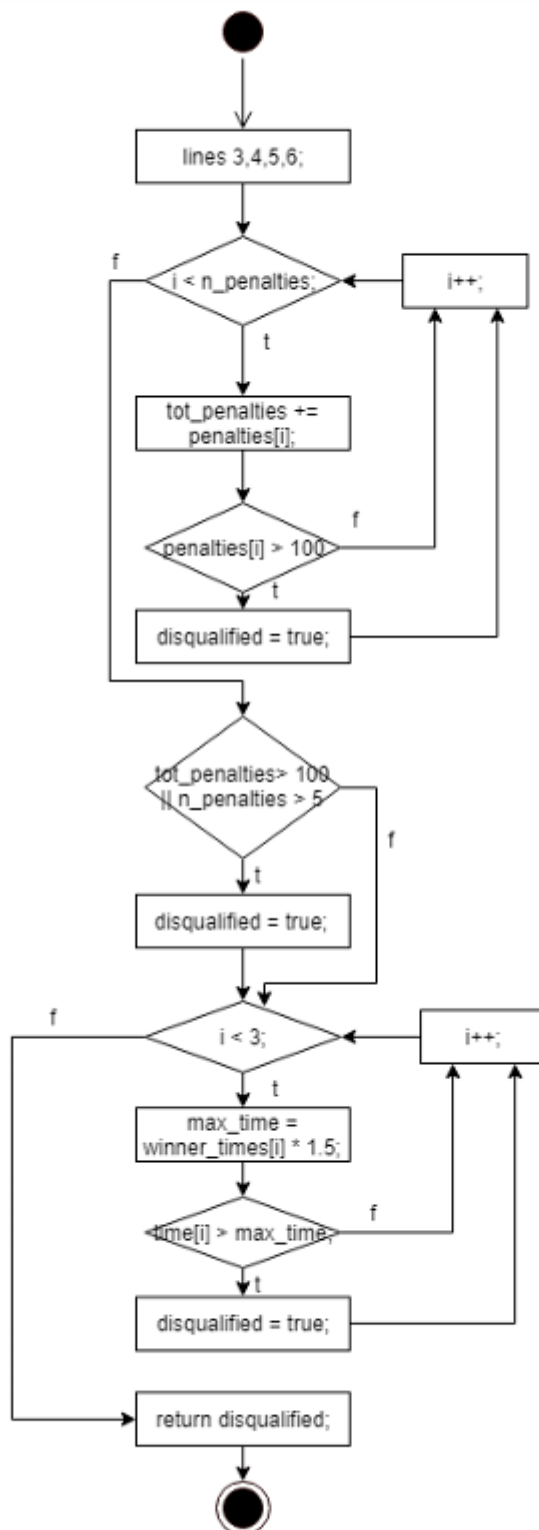
1  boolean racer_disqualified(int times[3],
    int winner_times[3], int n_penalties, int* penalties) {
2
3      bool disqualified = false;
4      int i;
5      int max_time;
6      int tot_penalties;
7
8      for (i = 0; i < n_penalties; i++) {
9          tot_penalties += penalties[i];
10         if (penalties[i] > 100)
11             disqualified = true;
12     }
13     if (tot_penalties > 100 || n_penalties > 5)
14         disqualified = true;
15
16     for (i=0; i<3; i++) {
17
18         max_time = winner_times[i] * 1.5;
19
20         if (time[i] > max_time[i])
21             disqualified = true;
22     }
23 }
24
25 }

```

| Coverage type              | Number of test cases needed to obtain 100% coverage | Coverage obtained with test cases defined (%)                           | Test cases defined                      |
|----------------------------|-----------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------|
| Node                       | 1                                                   | 100                                                                     | T1                                      |
| Edge                       | 2                                                   | 100                                                                     | T1, T2                                  |
| Multiple condition line 13 | 4                                                   | 100                                                                     | T1 (TF), T2 (FF), T3 (FT), T4 (TT)      |
| Loop line 8                | 3                                                   | 100                                                                     | T6 (0 loops), T5 (1 loop), T3 (n loops) |
| Path                       | $2 \cdot 2^3 \cdot 2^{(n\_penalties-1)}$            | 100% coverage obtainable with automated test case generation mechanisms | -                                       |

Write test case ID (T1, T2 ..) in the rightmost column, and test cases here

T1({10, 10, 10}, {20, 20, 20}, 2, {150, 50});  
T2({10, 10, 10}, {20, 20, 20}, 2, {5, 5});  
T3({10, 10, 10}, {20, 20, 20}, 6, {5, 5, 5, 5, 5, 5});  
T4({10, 10, 10}, {20, 20, 20}, 5, {50, 50, 50, 50, 50, 50});  
T5({10, 10, 10}, {20, 20, 20}, 1, {50});  
T6({10, 10, 10}, {20, 20, 20}, 0, {});



3 (7 points) – For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage. For the test cases, **write only the input value**.

Write control flow graph here

```

1  int grade(int answers[5][4], int solutions[5][4]) {
2
3      int grade = 0;
4      int i, j;
5
6      for (i = 0; i < 5; i++) {
7          for (j = 0; j < 4; j++) {
8              if (answers[i][j] == solutions[i][j] && solutions[i][j] == 1)
9                  grade+=3;
10             else if (answers[i][j] != solutions[i][j] && solutions[i][j] == 0)
11                 grade-=1;
12         }
13     }
14     if (grade < 0)
15         return 0;
16     else
17         return grade;
18 }

```

| Coverage type             | Number of test cases needed to obtain 100% coverage         | Coverage obtained with test cases defined (%)                  | Test cases defined |
|---------------------------|-------------------------------------------------------------|----------------------------------------------------------------|--------------------|
| Node                      | 2                                                           | 100%                                                           | T1, T2             |
| Edge                      | 2                                                           | 100%                                                           | T1, T2             |
| Multiple condition line 8 | In theory 4, 1 with different iterations of the inner loop. | 100%                                                           | T1                 |
| Loop line 6               | Not feasible: loop is not controlled by input values.       | 33% (only multiple iteration)                                  | T1 or T2           |
| Path                      | $3^{(4*5)} * 2$                                             | More than six million test cases to obtain full path coverage. |                    |

To avoid writing all the values for the input matrices (20 values) only the first rows of answers and solutions are written, and it is supposed that all other values are equal to 0.

T1({{1, 1, 0, 0, 0}, ...}, {{1, 0, 1, 0, 0}, ...}) -> grade = 1;

T2({{1, 1, 1, 1, 1}, ...}, {{0, 0, 0, 0, 0}, ...}) -> grade = 0 (grade less than 0 before line 14)

3 (7 points) – For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage. For the test cases, **write only the input value**.

Write control flow graph here

```

1 int canBePromoted(int grade1, int grade2, int grade3){
2     int grades[3];
3     grades[0] = grade1;
4     grades[1] = grade2;
5     grades[2] = grade3;
6     int countZero =0; int i=0;
7     int sum =0; float average;
8     for(i =0; i<3; i++){
9         if (grades[i] ==0) countZero++;
10        sum = sum + grades[i];
11    }
12    average = sum/(3-countZero) ;
13    if (countZero >1 || average <= 5.9 ) return 0;
14    return 1;
15 }

```

| Coverage type              | Number of test cases needed to obtain 100% coverage | Coverage obtained with test cases defined (%)            | Test cases defined |
|----------------------------|-----------------------------------------------------|----------------------------------------------------------|--------------------|
| Node                       | 2                                                   | 100%                                                     |                    |
| Edge                       | 2                                                   | 100%                                                     |                    |
| Multiple condition line 13 | 4                                                   | 100%                                                     |                    |
| Loop line 8                | 3                                                   | Only 1/3 possible                                        |                    |
| Path                       | 16<br>(8 in for line 8 times 2 line 13)             | Not all feasible, conditions in line 9 and 13 are linked |                    |

Write test case ID (T1, T2 ..) in the rightmost column, and test cases here

3 (7 points) – For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage. For the test cases, **write only the input value**.

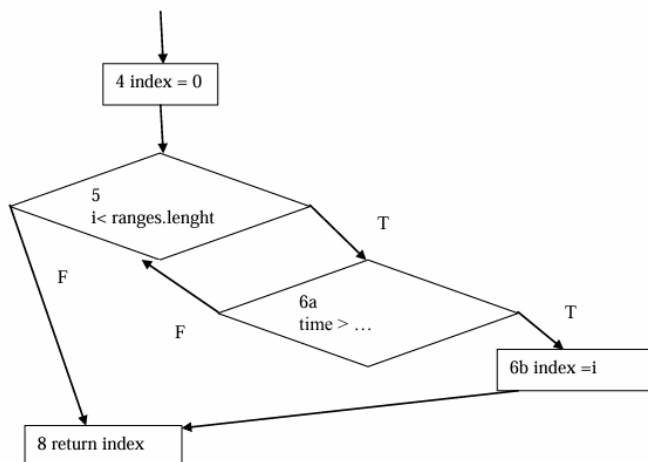
```

1  public int searchSpace(int space) {
2      int[] ranges = {0, 8, 24};
3
4      int index = 0;
5      for (int i=1; i< ranges.length; i++){
6          if (space > ranges[i-1] && space < ranges[i])
7              {index = i; break;}
8          }
9      return index;
10 }

```

write control flow here

| Coverage type                  | Feasibility (Y/N) | Coverage obtained (%)                                                                                                                                                                                                                                                                       | Test cases                                      |
|--------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Node                           | Y                 | 100%                                                                                                                                                                                                                                                                                        | T(12)                                           |
| Edge                           | Y                 | 100%                                                                                                                                                                                                                                                                                        | T(12)                                           |
| Multiple condition<br>(line 6) | Y                 | 75%<br>TT<br>TF<br>FT<br>FF                                                                                                                                                                                                                                                                 | T(1) or T(12)<br>T(12)<br>T(-6)<br>Not feasible |
| Loop                           | Y                 | The loop is executed at maximum 2 times (== ranges.length-1). There is no way not to enter the loop, nor to loop only once (it is possible to loop once, but using the break, and not the condition within the for)<br>1/3 max coverage possible (2/3 considering the loop once with break) | T(12)                                           |
| Path                           | Y                 | Only 3 paths are possible, so 100% path coverage is feasible:<br>4 5 6a 6b 8<br>4 5 6a 5 6a 6b 8<br>4 5 6a 5 6a 5 8                                                                                                                                                                         | T(1)<br>T(12)<br>T(30), or T(-6)                |



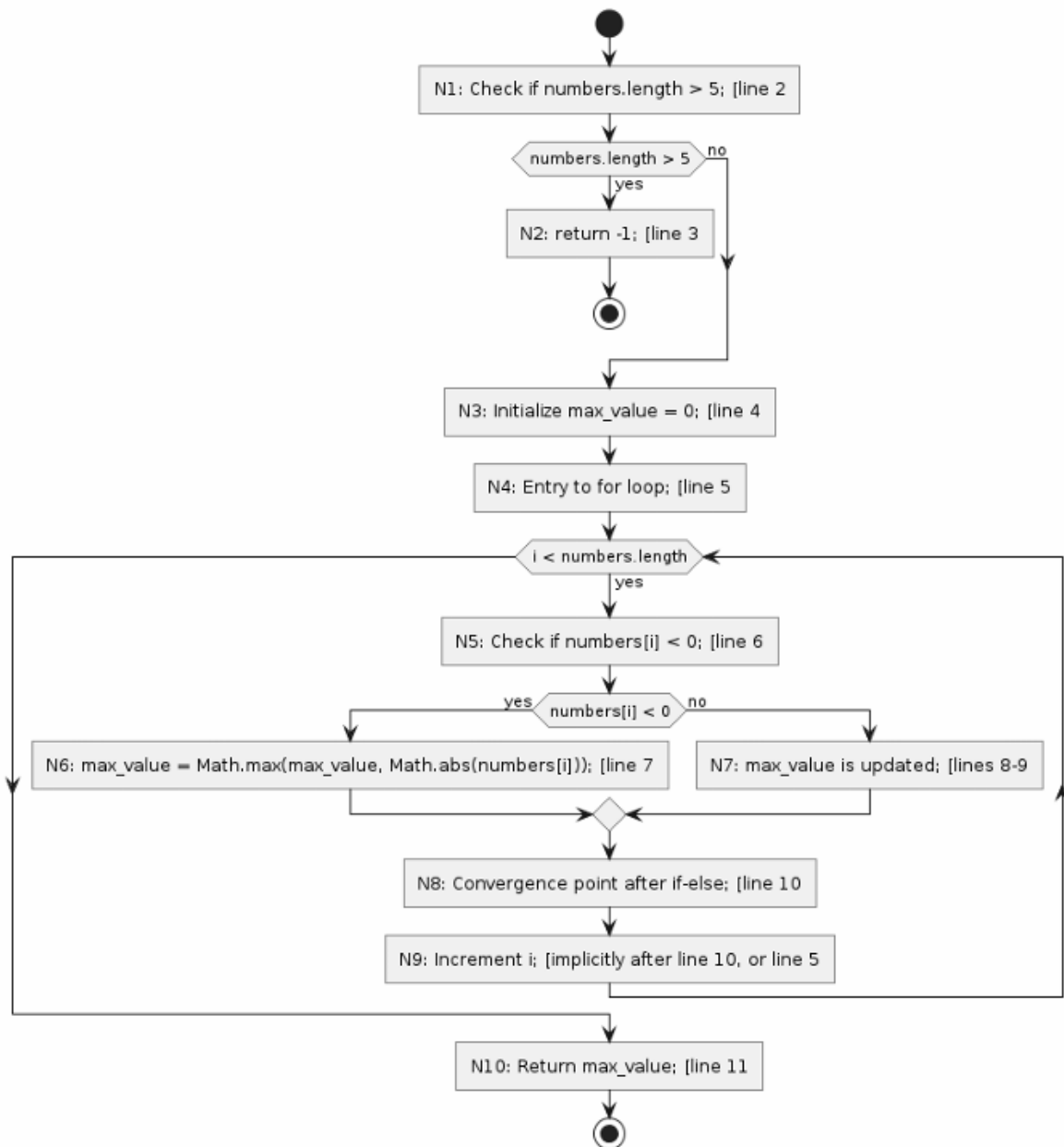
```

function max_absolute(numbers: number[]): number {
  if (numbers.length > 5) return -1;

  let max_value = 0;
  for (let i = 0; i < numbers.length; i++) {
    if (numbers[i] < 0)
      max_value = Math.max(max_value, Math.abs(numbers[i]));
    else
      max_value = Math.max(max_value, numbers[i]);
  }
  return max_value;
}

```

Control Flow Graph for max\_absolute Function





| Test case                               | Nodes covered       | Node coverage per TC | Edges covered                           | Edge coverage per TC | Loop (line 5) |
|-----------------------------------------|---------------------|----------------------|-----------------------------------------|----------------------|---------------|
| <b>all_equals</b><br>{0,0,0,0,0}        | 1 2 4 5<br>6 8 10   | 70 %                 | 1-2 2-4 4-5 5-6 6-8<br>8-5 5-10         | 70 %                 | 5 iter        |
| <b>all_positive</b><br>{1,2,3,4,5}      | 1 2 4 5<br>6 8 10   | 70 %                 | 1-2 2-4 4-5 5-6 6-8<br>8-5 5-10         | 70 %                 | 5 iter        |
| <b>all_negative</b><br>{-1,-2,-3,-4,-5} | 1 2 4 5<br>6 7 10   | 70 %                 | 1-2 2-4 4-5 5-6 6-7<br>7-5 5-10         | 70 %                 | 5 iter        |
| <b>out_of_size</b><br>{1,2,3,4,5,6}     | 1 2 3               | 30 %                 | 1-2 2-3                                 | 20 %                 | 0 iter        |
| <b>mixed</b><br>{-10,10,3,5,-6}         | 1 2 4 5<br>6 7 8 10 | 80 %                 | 1-2 2-4 4-5 5-6 6-7<br>7-5 6-8 8-5 5-10 | 90 %                 | 5 iter        |
| <b>empty</b> {}                         | 1 2 4 5<br>10       | 50 %                 | 1-2 2-4 4-5 5-10                        | 40 %                 | 0 iter        |
| <b>one</b> {1}                          | 1 2 4 5<br>6 8 10   | 70 %                 | 1-2 2-4 4-5 5-6 6-8<br>8-5 5-10         | 70 %                 | 1 iter        |

Node-coverage per test suite: 100 % (all 10 nodes executed across the suite)

Edge-coverage per test suite: 100 % (all 10 edges exercised)

## Exercise 2 – Convert Int

```
class ConvertInt {
    public convert(str: string[]): number | Error {
        if (str.length > 6)
            throw new Error("Invalid length");
        let number = 0, digit, i = 0;
        if (str[0] === '-')
            i = 1;
        for (; i < str.length; i++) {
            digit = str[i].charCodeAt(0) - '0'.charCodeAt(0);
            number = number * 10 + digit;
        }
        if (str[0] === '-')
            number = -number;
        if (number > 32767 || number < -32768)
            throw new Error("Number out of range");
        return number;
    }
}
```

| Test Case                                             | Statement                    | Decision         | Multiple condition | Loop coverage | Path coverage               |
|-------------------------------------------------------|------------------------------|------------------|--------------------|---------------|-----------------------------|
| <b>T-len</b><br>[ '1', '2', '3', '4', '5', '6', '7' ] | 3                            | D1 T             | —                  | 0             | P <sub>1</sub> length-error |
| <b>T-sign</b> [ '-' ]                                 | 3 7 14 15                    | D1 F, D2 T, D4 F | F    F             | 0             | P <sub>7</sub> only-sign    |
| <b>T-pos</b> [ '1', '2', '3' ]                        | 3 6 8 9 10<br>12 13 15       | D1 F, D2 F, D4 F | F    F             | 3             | P <sub>3</sub> normal-pos   |
| <b>T-neg</b> [ '-', '1', '2', '3' ]                   | 3 6 7 9 10<br>12 13 14<br>15 | D2 T, D4 F       | F    F             | 3             | P <sub>4</sub> normal-neg   |
| <b>T-ov+</b> [ '3', '3', '0', '0', '0' ]              | 3 6 8 9 10<br>12 13 14       | D4 T             | T    F             | 5             | P <sub>5</sub> overflow-pos |
| <b>T-ov-</b><br>[ '-', '3', '3', '0', '0', '0' ]      | 3 6 7 9 10<br>12 13 14       | D4 T             | F    T             | 5             | P <sub>6</sub> overflow-neg |

**Statement coverage: 100 %**

**Decision coverage: 100 %**

**Condition coverage: 100 %**

**Multiple condition coverage 75%**

**Loop coverage: 100 % (L1 exercised with 0 ≤ 1 iterations)**

**Path coverage: 6 actually reachable paths**

## Exercise 3 – EventsQueue

```
class EventsQueue {
    private queue: number[] = [];

    public insert(event: number): void {
        if (event <= 0) return;
        let left = 0, right = this.queue.length - 1;
        while (left <= right) {
            const mid = Math.floor((left + right) / 2);
            if (this.queue[mid] === event)
                return;
            if (this.queue[mid] < event) left = mid + 1;
            else right = mid - 1;
        }

        this.queue.splice(left, 0, event);
    }

    public pop(): number {
        if (this.queue.length === 0) return -1;
        return this.queue.shift();
    }

    public print(): void {
        console.log(this.queue.join(" "));
    }

    public size(): number {
        return this.queue.length;
    }
}
```

| Test case                   | Call sequence                                    | Statement coverage | Decision coverage                          | Multiple-condition coverage | Loop coverage            | Path Coverage |
|-----------------------------|--------------------------------------------------|--------------------|--------------------------------------------|-----------------------------|--------------------------|---------------|
| TC-1 non-positive           | <code>insert(0)</code><br>→ <code>size()</code>  | 2 / 12 ≈ 17 %      | 1 / 5 = 20 %<br>(D1 T)                     | n/a                         | while <b>not entered</b> | early-exit    |
| TC-2 first insert (+ print) | <code>insert(5)</code> ,<br><code>print()</code> | 5 / 12 ≈ 42 %      | 2 / 5 = 40 %<br>(D1 F, D2 F)               | n/a                         | 0 iterations             | insert-empty  |
| TC-3 duplicate              | <code>insert(5)</code> ,<br><code>size()</code>  | 6 / 12 = 50 %      | 3 / 5 = 60 %<br>(D1 F, D2 T, D3 T)         | n/a                         | 1 iteration              | duplicate     |
| TC-4 insert before first    | <code>insert(3)</code>                           | 6 / 12 = 50 %      | 4 / 5 = 80 %<br>(D1 F, D2 T/F, D3 F, D4 F) | n/a                         | 1 iteration              | insert-front  |
| TC-5 insert after last      | <code>insert(7)</code>                           | 6 / 12 = 50 %      | 4 / 5 = 80 %<br>(D1 F, D2 T/F, D3 F, D4 T) | n/a                         | ≥ 2 iterations           | insert-back   |
| TC-6 pop to empty           | <code>pop()</code> × 4                           | 4 / 12 ≈ 33 %      | 1 / 5 = 20 %<br>(D5 F→T)                   | n/a                         | –                        | pop-path      |

Statement coverage: 100 %

Branch/decision coverage: 100 %

Condition coverage: 100 %

Multiple condition coverage: N/A

Loop coverage: 100 % (zero, one, many)

Path coverage: six feasible paths

## Exercise 4 – Compute Fee

```
class Example5 {
    static computeFee(duration: number, minRate: number, minRate2: number): number {
        if (duration < 0 || minRate < 0 || minRate2 < 0) {
            return -1.0;
        }
        if (duration <= 30) {
            return 0.0;
        }
        if (duration <= 90) {
            return minRate * (duration - 30);
        }
        return 60 * minRate + (duration - 90) * minRate2;
    }
}
```

| Test Case            | Statement | Decision               | Multiple condition |
|----------------------|-----------|------------------------|--------------------|
| MC-1 (-1, 0, 0)      | S2        | D1 = T                 | T F F              |
| MC-2 (0, -1, 0)      | S2        | D1 = T                 | F T F              |
| MC-3 (0, 0, -1)      | S2        | D1 = T                 | F F T              |
| MC-4 (-1, -1, 0)     | S2        | D1 = T                 | T T F              |
| MC-5 (-1, 0, -1)     | S2        | D1 = T                 | T F T              |
| MC-6 (0, -1, -1)     | S2        | D1 = T                 | F T T              |
| MC-7 (-1, -1, -1)    | S2        | D1 = T                 | T T T              |
| MC-8 (5, 5, 5)       | S3 S4     | D1 = F, D2 = T, D3 = T | F F F, (T T)       |
| MC-9 (1, 1, 100)     | S3 S4     | D1 = F, D2 = T, D3 = F | F F F, (T F)       |
| MC-10 (100, 100, 50) | S3 S4     | D1 = F, D2 = F         | F F F, (F T)       |
| MC-11 (1, 1, 120)    | S3 S4     | D1 = F, D2 = F         | F F F, (F F)       |

Statement coverage: 100 %

Decision coverage: 100 %

Multiple-condition coverage: 100 % (8/8 combos for D1, 4/4 combos for D2^D3)

## Exercise 5 – Compute Fee 2

```
class Example6 {
    static computeFee(basePrice: number, n_passengers: number, n_over18: number, n_under15: number): number {
        if (basePrice < 0 || n_passengers < 2 || n_passengers > 5 || n_over18 < 0 || n_under15 < 0) {
            return -1.0;
        }
        if (n_over18 + n_under15 > n_passengers) {
            return -1.0;
        }
        if (n_passengers >= 2 && n_passengers <= 5 && n_over18 > 0) {
            return basePrice * (n_passengers - n_under15);
        } else {
            return basePrice * n_passengers;
        }
    }
}
```

| Test Case       | Statement | Decision | Multiple-condition |
|-----------------|-----------|----------|--------------------|
| T1 (-1, 1, 1)   | 3 4       | T--      | TFF                |
| T2 (10, -1, 1)  | 3 4       | T--      | FTF                |
| T3 (10, 1, -1)  | 3 4       | T--      | FFT                |
| T4 (-1, -1, 1)  | 3 4       | T--      | TTF                |
| T5 (-1, 1, -1)  | 3 4       | T--      | TFT                |
| T6 (10, -1, -1) | 3 4       | T--      | FTT                |
| T7 (-1, -1, -1) | 3 4       | T--      | TTT                |
| T8 (20, 1, 1)   | 3 6 7     | FT-      | FFF                |
| T9 (60, 2, 1)   | 3 9 10    | FFT      | FFF                |
| T10 (100, 2, 3) | 3 9 12    | FFF      | FFF                |

**Statement Coverage: 100%**

**Decision Coverage: 100%**

**Multiple-condition Coverage: 26,7% (8/30)**

## Exercise 6 – acceptableToEat

```
class Example4 {  
    static acceptableToEat(carb: number, protein: number, fat: number): boolean {  
        if (carb < 0 || protein < 0 || fat < 0) {  
            return false;  
        }  
        const totalCalories = 4 * carb + 4 * protein + 9 * fat;  
        const ratio = (carb + protein) / fat;  
        return totalCalories < 1000 && ratio > 0.5;  
    }  
}
```

| Test Case         | Statement | Decision       | Multiple condition |
|-------------------|-----------|----------------|--------------------|
| T1 (-1, 3, 1, 1)  | 3 4       | D1 T           | T F F F F          |
| T2 (10, 1, 0, 0)  | 3 4       | D1 T           | F T F F F          |
| T3 (10, 6, 1, 0)  | 3 4       | D1 T           | F F T F F          |
| T4 (10, 3, -1, 0) | 3 4       | D1 T           | F F F T F          |
| T5 (10, 3, 0, -1) | 3 4       | D1 T           | F F F F T          |
| T6 (10, 4, 3, 2)  | 3 6 7     | D1 F D2 T      | F F F F F          |
| T7 (10, 4, 2, 1)  | 3 9 10    | D1 F D2 F D3 T | F F F F F          |
| T8 (10, 3, 0, 3)  | 3 9 12    | D1 F D2 F D3 F | F F F F F          |

**Total statement coverage: 100.0%**

**Total decision coverage: 100.0%**

**Total condition coverage: 100.0%**

**Total multiple condition coverage: 75%**

# Exercise 7 – Elonian Tax Calculator

```
class Example7 {
  static elonianTaxCalculator(income: number, nDependents: number): number {
    if (income < 0) {
      console.log("You cannot have a negative income.\n");
      return -1;
    }
    if (nDependents <= 0) {
      console.log("You must have at least one dependent.\n");
      return -2;
    }

    let TaxSubTotal: number;
    if (income < 10000) {
      TaxSubTotal = 0.02 * income;
    } else if (income < 50000) {
      TaxSubTotal = 200 + 0.03 * (income - 10000);
    } else {
      TaxSubTotal = 1400 + 0.04 * (income - 50000);
    }

    let Exemption = nDependents * 50;
    let TaxTotal = TaxSubTotal - Exemption;

    if (TaxTotal < 0) {
      TaxTotal = 0;
    }

    console.log("Elbonian Tax Collection Agency \n");
    console.log("Tax Bill \n");
    console.log("Citizen's Income: " + income + '\n');
    console.log("Tax Subtotal: " + TaxSubTotal + '\n');
    console.log("Number of Dependents: " + nDependents + '\n');
    console.log("Tax Exemption: " + Exemption + '\n');
    console.log("Final Tax Bill: " + TaxTotal + '\n');

    return TaxTotal;
  }
}
```

| Test Case       | Statement                  | Decision          | Multiple condition | Path coverage                                 |
|-----------------|----------------------------|-------------------|--------------------|-----------------------------------------------|
| T1 (-1, 2)      | 3 4 5                      | T / - / - / - / - | -                  | P <sub>1</sub> early "income < 0"             |
| T2 (10 000, 0)  | 7 8 9                      | F / T / - / - / - | -                  | P <sub>2</sub> early "no dependents"          |
| T3 (9 000, 2)   | 12 13 14 21 22 25 29-35    | F / F / T / - / F | -                  | P <sub>3</sub> bracket < 10 000, tax positive |
| T4 (3 000, 100) | 12 13 14 21 22 25 26 29-35 | F / F / T / - / T | -                  | P <sub>4</sub> bracket < 10 000, tax floor 0  |
| T5 (20 000, 2)  | 12 15 16 21 22 25 29-35    | F / F / F / T / F | -                  | P <sub>5</sub> bracket 10 k–50 k              |
| T6 (60 000, 2)  | 12 17 18 21 22 25 29-35    | F / F / F / F / F | -                  | P <sub>6</sub> bracket > 50 k                 |

Total statement coverage: 100.0%

Total decision coverage: 100.0%

Total condition coverage: 100.0%

Total multiple condition coverage: N/A%

Total path coverage: 100.0% all six feasible execution paths

## Exercise 8 – foo() and bug()

```
function foo(a: number, b: number, c: number, d: number, e: number): number {  
  if (a === 0) {  
    return 0;  
  }  
  let x = 0;  
  if (a === b || (c === d && bug(a))) {  
    x = 1;  
  }  
  e = 1 / x;  
  return e;  
}  
  
function bug(a: number): boolean {  
  return a === 1;  
}
```

| Test Case          | Statement          | Decision           | Multiple condition | Path coverage                       |
|--------------------|--------------------|--------------------|--------------------|-------------------------------------|
| T1 (0, 0, 0, 0, 0) | 2 3                | T / –              | –                  | P1 – early return                   |
| T2 (1, 1, 2, 3, 5) | 2 5 6 7 9 10       | F / T (via A)      | T x x              | P2 – a===b branch                   |
| T3 (1, 2, 3, 3, 5) | 2 5 6 7 9 10<br>14 | F / T (via B<br>C) | F T T              | P3 – c===d && bug(a)<br>true        |
| T4 (2, 3, 4, 5, 5) | 2 5 6 9 10         | F / F              | F F x              | P4 – divide by 0 (bug not<br>eval.) |
| T5 (2, 3, 4, 4, 5) | 2 5 6 9 10 14      | F / F              | F T F              | P4 – divide by 0 (bug F)            |

Total statement coverage: 100.0%

Total decision coverage: 100.0%

Total condition coverage: 100.0%

Total multiple condition coverage: 75%

Total path coverage: 100.0% all four feasible execution paths